

DVSA Vulnerability Discovery & Remediation

ICS-344 — Information Security

Term 252 ■ Team 2 ■ Sections 1 & 2

Husain Al Muallim ■ Ali Al Qatari ■ Abdulwahed Zuhair Adi
202163730 ■ 202279780 ■ 202256380

May 2026

Agenda

- 1 Project Overview
- 2 Lessons 1–3: Injection, Auth, Data Exposure
- 3 Lessons 4–6: Cloud Config, Access Control, DoS
- 4 Lessons 7–10: Privilege, Logic, Deps, Exceptions
- 5 Key Takeaways

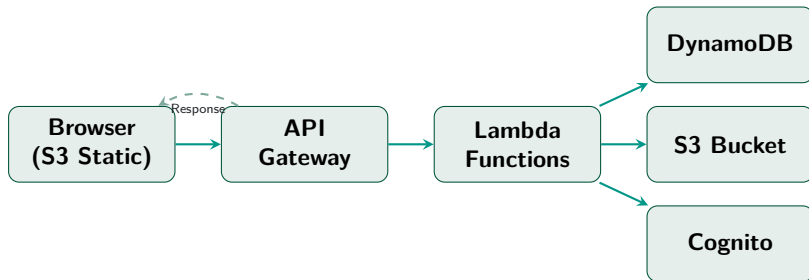
✿ Scope

- Deploy OWASP DVSA in a non-production AWS account
- Exploit **10 official vulnerability lessons**
- Apply remediations and verify each fix
- Document with structured 10-part analysis

✦ Deployment

- **Region:** us-east-1
- **Stack:** serverlessrepo-OWASP-DVSA
- **Services:** S3, API Gateway, Lambda, DynamoDB, SQS, Cognito, IAM, SES, CloudWatch, CloudTrail

Architecture & Request Pipeline



→ Pipeline

Browser → API Gateway → Lambda (e.g., ORDER-MANAGER) → DynamoDB / S3 / Cognito → Response



Tools Used

- curl, jq, Python requests
- Browser DevTools (Network, Console)
- AWS Console (Lambda, IAM, S3, API GW)
- AWS CLI, IAM Policy Simulator
- CloudWatch Logs, CloudTrail

✓ Process

- ① Validate environment end-to-end
- ② Identify & reproduce vulnerability
- ③ Capture evidence (screenshots, logs)
- ④ Apply targeted fix at root cause
- ⑤ Re-run exploit to verify fix
- ⑥ Confirm normal workflow intact

Lesson 01: Event Injection

★ Vulnerability

Type: Insecure Deserialization / RCE

Component: DVSA-ORDER-MANAGER

Root Cause

`node-serialize.unserialize()` uses `eval()` internally, executing attacker-controlled code from request body.

➤ **Takeaway:** Never use libraries that evaluate user input as code. Use strict, safe parsers for all untrusted data.

✗ Exploit

Crafted JSON with `__$ND_FUNC__$` tag executed arbitrary JS on Lambda, writing/reading files in `/tmp`.

✓ Fix & Verification

Replaced `unserialize()` with `JSON.parse()`.
Post-fix: same payload no longer executes;
CloudWatch clean.

Lesson 02: Broken Authentication

★ Vulnerability

Type: Missing API Gateway Authorizer

Component: DVSA-ORDER-MANAGER

📎 Root Cause

API Gateway /order POST had no Cognito Authorizer. Any JWT—valid or forged—was forwarded to Lambda.

✗ Exploit

Regular user called admin-orders action and received all users' order data without admin privileges.

✓ Fix & Verification

Attached Cognito Authorizer to API Gateway + added is_admin check in Lambda. Post-fix: returns 401/403.

➤ **Takeaway:** Enforce auth at the edge (API Gateway) *and* in code. Defense in depth prevents single-layer failures.

Lesson 03: Sensitive Data Exposure

★ Vulnerability

Type: Missing Authorization in Lambda
Component: DVSA-ADMIN-GET-RECEIPT

Root Cause

Lambda had zero auth checks. Any caller could invoke it directly and receive a signed S3 URL for all receipts.

➤ **Takeaway:** “Admin” in the name doesn’t enforce access. Lambda functions can bypass API Gateway; auth must be in-function.

✗ Exploit

Invoked Lambda via Console test with arbitrary date params; received signed URL to download all customer receipts.

✓ Fix & Verification

Added requestContext check + admin ARN verification. Post-fix: returns “Access denied” (1.6ms vs 4.3s).

Lesson 04: Insecure Cloud Configuration

★ Vulnerability

Type: Open S3 Bucket

Component: dvsa-receipts-bucket

Root Cause

S3 bucket deployed without “Block Public Access.” Broad IAM policies allowed external principals to write objects.

➤ **Takeaway:** Always enable Block Public Access unless explicitly justified. Default-open CloudFormation configs cause breaches.

✗ Exploit

Used `aws s3 cp` with a “hacker” profile to upload arbitrary files into the receipts bucket successfully.

✓ Fix & Verification

Enabled **Block All Public Access** on S3 bucket. Post-fix: upload returns Access Denied; receipts still work.

Lesson 05: Broken Access Control

★ Vulnerability

Type: IDOR — No Ownership Check

Component: DVSA-ORDER-MANAGER

Root Cause

Lambda extracted `order-id` from body without verifying the JWT user owns that order in DynamoDB.

✗ Exploit

User B sent `curl` with User C's order ID and received C's full order details (address, items, payment).

✓ Fix & Verification

Added `userOwnsOrder()` check before dispatching. Post-fix: returns "could not find order" for non-owners.

➤ **Takeaway:** APIs must enforce resource ownership server-side. Front-end restrictions alone are trivially bypassed.

Lesson 06: Denial of Service

★ Vulnerability

Type: No Rate Limiting on API Gateway

Component: DVSA-PAYMENT-PROCESSOR

Root Cause

No throttling on API Gateway; account had only 10 concurrent Lambda slots. Easily saturated by parallel requests.

✗ Exploit

Python threading script sent 15+ parallel billing requests; legitimate users got connection resets and 500 errors.

✓ Fix & Verification

Enabled API Gateway stage throttling (Rate=10/s, Burst=2). Post-fix: excess requests rejected at gateway layer.

► **Takeaway:** Availability is a security property. Serverless concurrency is finite—rate-limit critical endpoints like billing.

Lesson 07: Over-Privileged Function

★ Vulnerability

Type: Excessive IAM Permissions

Component: SendReceiptFunctionRole

✗ Exploit

Policy Simulator confirmed: s3:PutObject, dynamodb:Scan, all SES actions were **allowed**.

📝 Root Cause

Role had AmazonSESFullAccess + full S3 + full DynamoDB. A compromised Lambda would inherit all capabilities.

✓ Fix & Verification

Replaced 3 broad policies with 1 scoped inline policy (S3 GetObject + SES SendEmail only). Simulator: denied.

➤ **Takeaway:** Apply least privilege always. Use CloudTrail Generate Policy to right-size permissions based on actual usage.

Lesson 08: Logic Vulnerabilities

★ Vulnerability

Type: Race Condition in Order Update

Component: DVSA-ORDER-UPDATE

Root Cause

Status check used > 110 instead of > 100 , allowing modification during billing window (status 100).

✗ Exploit

Paid for cheap items, then updated order to expensive items before status advanced past 100. "Cart updated" returned.

✓ Fix & Verification

Changed threshold to > 100 . Post-fix: updates on non-open orders return "order already paid or in progress."

➤ **Takeaway:** Logic flaws evade scanners. State-machine transitions must be strict; review boundary conditions carefully.

Lesson 09: Vulnerable Dependencies

★ Vulnerability

Type: Known CVE in node-serialize

Component: DVSA-ORDER-MANAGER

Root Cause

`package.json` included `node-serialize v0.0.4`—CVE-2017-5941, CVSS 9.8 Critical (RCE via `eval`).

➤ **Takeaway:** Audit dependencies regularly (`npm audit`, Snyk). A single vulnerable package can compromise everything.

✗ Exploit

NVD lookup confirmed critical severity. The package was actively used for deserialization (see Lesson 01).

✓ Fix & Verification

Removed `node-serialize` from `package.json`; code already uses `JSON.parse()`. Lambda deploys cleanly.

Lesson 10: Unhandled Exceptions

★ Vulnerability

Type: Stack Trace Leakage

Component: DVSA-ORDER-GET

📝 Root Cause

Missing try/except: a malformed request raised `KeyError`, returning full traceback with file paths to client.

✗ Exploit

Sent `{"action": "get"}` without `order-id`; response contained `/var/task/get_order.py` and line numbers.

✓ Fix & Verification

Wrapped handler in try/except with `safe_error()` returning generic messages. Internals logged server-side only.

➤ **Takeaway:** Never expose stack traces to clients. Catch all exceptions at the handler boundary and return sanitized errors.

Security Themes Across All Lessons

✦ Defense in Depth

Security at multiple layers: API Gateway auth, Lambda code validation, IAM policies, and cloud resource config.

🔗 Least Privilege

IAM roles scoped to exact actions needed. Over-privileged roles amplify the blast radius of any compromise.

✎ Input Validation

All user input is untrusted. Use safe parsers, validate schemas, and never evaluate input as code.

✿ Dependency Hygiene

Regularly audit packages for CVEs. Remove unused libraries. Minimize attack surface.

★ Safe Error Handling

Never expose internals to clients. Log verbose details server-side; respond with generic messages.

➔ Availability Controls

Rate limiting and throttling protect shared serverless resources from exhaustion attacks.

Vulnerability Summary Matrix

#	Lesson	Vulnerability	Component	Fix
01	Event Injection	Insecure deserialization	ORDER-MANAGER	Replace with JSON.parse()
02	Broken Auth	Missing authorizer	API Gateway	Attach Cognito Authorizer
03	Data Exposure	No auth in Lambda	ADMIN-GET-RECEIPT	Add requestContext + ARN check
04	Cloud Config	Open S3 bucket	Receipts bucket	Block All Public Access
05	Access Control	IDOR / no ownership	ORDER-MANAGER	userOwnsOrder() check
06	DoS	No rate limiting	API Gateway	Stage-level throttling
07	Over-Privilege	Excessive IAM	SendReceipt role	Scoped inline policy
08	Logic Flaw	Race condition	ORDER-UPDATE	Status threshold >100
09	Vuln Deps	CVE-2017-5941	package.json	Remove node-serialize
10	Exceptions	Stack trace leak	ORDER-GET	try/except + safe_error()

Thank You

Husain Al Muallim ■ Ali Al Qatari ■ Abdulwahed Zuhair Adi

ICS-344 — Term 252 — Team 2